



# Curso Javascript básico

## Aula 08 - Trabalhando com Números e o Objeto Math



# Tipos de números

Usamos diferentes termos para descrever diferentes tipos de números decimais, por exemplo:

**Integers (inteiros)** são números inteiros, ex. 10, 400 ou -5.

**Números de ponto flutuante (floats)** tem pontos e casas decimais, por exemplo 12.5 e 56.7786543.

**Doubles** são tipos de floats que tem uma precisão maior do que os números de ponto flutuante padrões (significando que eles são mais precisos, possuindo uma grande quantidade de casas decimais).



# Tipos de números

Temos também diferentes tipos de sistemas numéricos!

O decimal tem por base 10 (o que significa que ele usa um número entre 0–9 em cada casa), mas temos também algo como:

**Binário** — A linguagem de menor level dos computadores; 0s e 1s.

**Octal** — Base 8, usa um caractere entre 0–7 em cada coluna.

**Hexadecimal** — Base 16, usa um caractere entre 0–9 e depois um entre a–f em cada coluna. Você pode já ter encontrado esses números anteriormente, trabalhando com cores no CSS.



# Tipos de números

Temos também diferentes tipos de sistemas numéricos!

O decimal tem por base 10 (o que significa que ele usa um número entre 0–9 em cada casa), mas temos também algo como:

**Binário** — A linguagem de menor level dos computadores; 0s e 1s.

**Octal** — Base 8, usa um caractere entre 0–7 em cada coluna.

**Hexadecimal** — Base 16, usa um caractere entre 0–9 e depois um entre a–f em cada coluna. Você pode já ter encontrado esses números anteriormente, trabalhando com cores no CSS.



# Tipos de números

O objeto JavaScript Number é um objeto encapsulado que permite você trabalhar com valores numéricos. Um objeto Number é criado utilizando o construtor Number().

*`new Number(value);`*

*Onde o value e o valor numérico do objeto sendo criado.*



# Tipos de números

## Descrição

Os principais usos para o objeto Number são:

Se o objeto não pode ser convertido para um número, é retornado NaN.

Fora do contexto de um construtor (Ex., Sem o operador new, Number pode ser utilizado para realizar uma conversão de tipo.



# Propriedades

| Nome                     | Descrição                                                                                                          |
|--------------------------|--------------------------------------------------------------------------------------------------------------------|
| Number.EPSILON           | O menor intervalo entre dois números representáveis.                                                               |
| Number.MAX_SAFE_INTEGER  | O inteiro máximo seguro em JavaScript ( $2^{53} - 1$ )                                                             |
| Number.MAX_VALUE         | O maior número representável positivo.                                                                             |
| Number.MIN_SAFE_INTEGER  | O inteiro mínimo seguro em JavaScript ( $-(2^{53} - 1)$ ).                                                         |
| Number.MIN_VALUE         | O número mínimo representável positivo - isto é, o número positivo mais próximo de zero (sem ser zero na verdade). |
| Number.NaN               | Valor especial que não é número.                                                                                   |
| Number.NEGATIVE_INFINITY | Valor especial representando infinito negativo; retornado no "overflow".                                           |
| Number.POSITIVE_INFINITY | Valor especial representando infinito; retornado no "overflow".                                                    |
| Number.prototype         | Permite a adição de propriedades a um objeto Number.                                                               |



# Métodos

| Nome                   | Descrição                                                                                            |
|------------------------|------------------------------------------------------------------------------------------------------|
| Number.isNaN()         | Determina se o valor passado é NaN.                                                                  |
| Number.isFinite()      | Determina se o tipo e o valor passado é um número finito.                                            |
| Number.isInteger()     | Determina se o tipo do valor passado é inteiro.                                                      |
| Number.isSafeInteger() | Determina se o tipo do valor passado é um inteiro seguro (número entre $-(2^{53}-1)$ e $2^{53}-1$ ). |
| Number.parseFloat()    | O valor é o mesmo que parseFloat do objeto global.                                                   |
| Number.parseInt()      | O valor é o mesmo que parseInt do objeto global.                                                     |





# Objeto Math

## Matemática básica no JavaScript, números e operadores

Muito do que fazemos se baseia no processamento de dados numéricos, cálculo de novos valores, etc. O JavaScript tem uma configuração completa de funções matemáticas disponíveis.

O JavaScript tem apenas um tipo de dados para números, o Number. Isso significa que qualquer que seja o tipo de números com os quais lidamos em JavaScript, manipulamos do mesmo jeito.



# Objeto Math

## Operadores aritméticos

| Operador | Nome              | Propósito                                                                                    | Exemplo                                                                   |
|----------|-------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| +        | Adição            | Adiciona um número a outro.                                                                  | 6 + 9                                                                     |
| -        | Subtração         | Subtrai o número da direita do número da esquerda.                                           | 20 - 15                                                                   |
| *        | Multiplicação     | Multiplica um número pelo outro.                                                             | 3 * 7                                                                     |
| /        | Divisão           | Divide o número da esquerda pelo número da direita.                                          | 10 / 5                                                                    |
| %        | Restante (modulo) | Retorna o resto da divisão em números inteiros do número da esquerda pelo número da direita. | 8 % 3 (retorna 2; como três cabe duas vezes em 8, deixando 2 como resto.) |



# Objeto Math

## Precedência de operador

A precedência de operadores em JavaScript é semelhante ao ensinado nas aulas de matemática na escola — Multiplicação e divisão são realizados primeiro, depois a adição e subtração (a soma é sempre realizada da esquerda para a direita).

$$(num2 + num1) / (8 + 2);$$



# Objeto Math

## Operadores de incremento e decremento

Às vezes você desejará adicionar ou subtrair, repetidamente, um valor de uma variável numérica. Convenientemente isto pode ser feito usando os operadores incremento ++ e decremento --. Usamos ++



# Métodos

## método

`abs(x)`

`acos(x)`

`acosh(x)`

`asin(x)`

`asinh(x)`

`atan(x)`

`atan2(y, x)`

`atanh(x)`

`cbrt(x)`

`ceil(x)`

`clz32(x)`

`cos(x)`

`cosh(x)`

`E`

`exp(x)`

## Descrição

Retorna o valor absoluto de x

Retorna o arco-cosseno de x, em radianos

Retorna o arco-cosseno hiperbólico de x

Retorna o arco-seno de x, em radianos

Retorna o arco seno hiperbólico de x

Retorna o arco tangente de x como um valor numérico entre  $-\pi/2$  e  $\pi/2$  radianos

Retorna o arco tangente do quociente de seus argumentos

Retorna o arco tangente hiperbólico de x

Retorna a raiz cúbica de x

Retorna x, arredondado para cima para o inteiro mais próximo

Retorna o número de zeros à esquerda em uma representação binária de 32 bits de x

Retorna o cosseno de x (x está em radianos)

Retorna o cosseno hiperbólico de x

Retorna o número de Euler (aprox. 2,718)

Retorna o valor de  $E^x$



# Métodos

## método

expm1(x)

floor(x)

fround(x)

LN2

LN10

log(x)

log10(x)

LOG10E

log1p(x)

log2(x)

LOG2E

max(x1,x2,...)

min(x1,x2,...)

PI

## Descrição

Retorna o valor de  $e^x$  menos 1

Retorna x, arredondado para baixo para o inteiro mais próximo

Retorna a representação float mais próxima (precisão simples de 32 bits) de um número

Retorna o logaritmo natural de 2 (aprox. 0,693)

Retorna o logaritmo natural de 10 (aprox. 2,302)

Retorna o logaritmo natural de x

Retorna o logaritmo de base 10 de x

Retorna o logaritmo de base 10 de E (aprox. 0,434)

Retorna o logaritmo natural de  $1 + x$

Retorna o logaritmo de base 2 de x

Retorna o logaritmo de base 2 de E (aprox. 1,442)

Retorna o número com o maior valor

Retorna o número com o menor valor

Retorna PI (aprox. 3,14)



# Métodos

## método

`pow(x, y)`

`random()`

`round(x)`

`sign(x)`

`sin(x)`

`sinh(x)`

`sqrt(x)`

`SQRT1_2`

`SQRT2`

`tan(x)`

`tanh(x)`

`trunc(x)`

## Descrição

Retorna o valor de x elevado a y

Retorna um número aleatório entre 0 e 1

Arredonda x para o inteiro mais próximo

Retorna o sinal de um número (verifica se é positivo, negativo ou zero)

Retorna o seno de x (x está em radianos)

Retorna o seno hiperbólico de x

Retorna a raiz quadrada de x

Retorna a raiz quadrada de 1/2 (aprox. 0,707)

Retorna a raiz quadrada de 2 (aprox. 1,414)

Retorna a tangente de um ângulo

Retorna a tangente hiperbólica de um número

Retorna a parte inteira de um número (x)